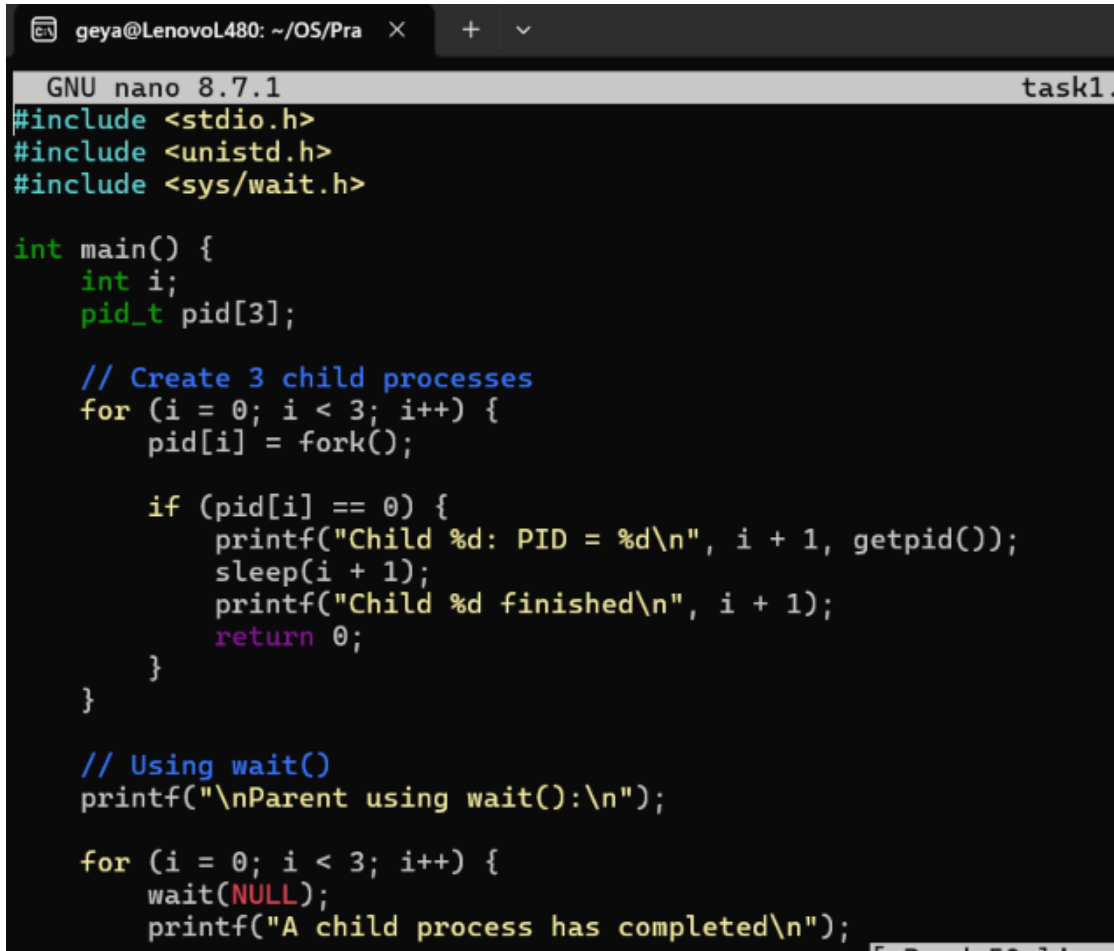


Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions

Code:



The screenshot shows a terminal window with a nano editor editing a file named 'task1.c'. The terminal title bar indicates the user is 'geya' on a 'LenovoL480' machine, with the current directory being '~/OS/Pra'. The nano editor's status bar shows 'GNU nano 8.7.1' and the filename 'task1.c'. The code in the editor is a C program that creates three child processes using 'fork()' and synchronizes their completion using 'wait()'. The program includes headers for standard I/O, unistd, and sys/wait. In the 'main' function, it declares an integer 'i' and an array of 'pid_t' values 'pid' of size 3. It then enters a loop to create three child processes. Each child process prints its PID, sleeps for its index (0, 1, or 2 seconds), and then prints a message indicating it has finished. The parent process then uses 'wait(NULL)' to wait for each child to complete, printing a message for each one.

```
geya@LenovoL480: ~/OS/Pra  ×  +  v
GNU nano 8.7.1 task1.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int i;
    pid_t pid[3];

    // Create 3 child processes
    for (i = 0; i < 3; i++) {
        pid[i] = fork();

        if (pid[i] == 0) {
            printf("Child %d: PID = %d\n", i + 1, getpid());
            sleep(i + 1);
            printf("Child %d finished\n", i + 1);
            return 0;
        }
    }

    // Using wait()
    printf("\nParent using wait():\n");

    for (i = 0; i < 3; i++) {
        wait(NULL);
        printf("A child process has completed\n");
    }
}
```

```

// Create 3 more children
for (i = 0; i < 3; i++) {
    pid[i] = fork();

    if (pid[i] == 0) {
        printf("Child %d: PID = %d\n", i + 1, getpid());
        sleep(i + 1);
        printf("Child %d finished\n", i + 1);
        return 0;
    }
}

// Using waitpid()
printf("\nParent using waitpid():\n");

for (i = 0; i < 3; i++) {
    waitpid(pid[i], NULL, 0);
    printf("Child with PID %d has completed\n", pid[i]);
}

return 0;

```

Output:

```

geya@LenovoL480:~/OS/Practical/Week 4$ ./task1
Child 1: PID = 564

Child 2: PID = 565
Parent using wait():
Child 3: PID = 566
Child 1 finished
A child process has completed
Child 2 finished
A child process has completed
Child 3 finished
A child process has completed
Child 1: PID = 567
Child 2: PID = 568

Parent using waitpid():
Child 3: PID = 569
Child 1 finished
Child with PID 567 has completed
Child 2 finished
Child with PID 568 has completed
Child 3 finished
Child with PID 569 has completed
geya@LenovoL480:~/OS/Practical/Week 4$ |

```